DEEP LEARNING — 046211
FINAL EXAM A
30/4/2022

EXAM DURATION: 3 HOURS

INSTRUCTIONS.

1. You can use a calculator and the 2 formula sheets (4 pages).
2. You need to answer all question parts, unless you received a special permit.
3. The weight of each question is written in the form. The total number of points is 100 points.
4. If you received a special permit (due to reserve duty) you can choose to answer only two questions out of three. In that case, the questions points will be normalized so the total number of points will remain 100.
5. Please write clearly and orderly. You can write either in English or in Hebrew.
6. You must give the full details of the solution derivation. Solutions without explanations will not be given a score, unless written otherwise.
7. Once you finish the exam, you need to use gradescope to upload your exam. In addition, you need to submit the notebook.

1. Transformers

For a series of T values, each with d features $X \in \mathbb{R}^{T\times d}$, a Self-Attention (SA) layer with parameters

$$\mathbf{W}_Q \in \mathbb{R}^{d\times D}, \mathbf{W}_K \in \mathbb{R}^{d\times D}, \mathbf{W}_V \in \mathbb{R}^{d\times M}$$

is defined as:

$$(1)\quad \mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \mathbf{K} = \mathbf{X}\mathbf{W}_K, \mathbf{V} = \mathbf{X}\mathbf{W}_V$$

$$(2)\, \mathrm{SA}(\mathbf{X}) = \mathbf{V}' = \mathrm{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{D}}\right)\mathbf{V}$$

We will now look at an Encoder-Decoder Transformer

1. Give an example usage for this kind of architecture.

2. Draw the architecture and shortly explain each of the following components: Positional Encoding, AddNorm, Feed Forward.

3. Explain the difference between the Encoder and Decoder components.

One of the issues in computing SA is the computation speed and the memory required for this computation.

4. How many multiplications are required for calculation of a single SA layer, as defined in Eq. 2 (without the computation needed for Eq. 1, and other non-multiplication operations).

5. We will now see how to make this computation more efficient. We define a similarity function between the vectors q and k as follows:

$$\mathrm{sim}(\mathbf{q}, \mathbf{k}) = \exp\left(\frac{\mathbf{q}^T\mathbf{k}}{\sqrt{D}}\right).$$

Show that the expression for the i^{th} value in the series ($1 \le i \le T$):

$$(3) V_i' = \frac{\sum_{j=1}^{T} \mathrm{sim}(\mathbf{Q}_i, \mathbf{K}_j)V_j}{\sum_{r=1}^{T} \mathrm{sim}(\mathbf{Q}_i, \mathbf{K}_r)}$$

is equivalent to Eq. 2 above, where a matrix with index i denotes the i^{th} row in the matrix. For example, Q_i denotes the vector of the i^{th} row of the matrix Q.

6. Assuming we can write the similarity function as

$$\mathrm{sim}(\mathbf{q}, \mathbf{k}) = \phi(\mathbf{q})^\top \phi(\mathbf{k})$$

show how we can re-write (3) while using the associativity rule for matrices

$$\left(\phi(\mathbf{Q})\,\phi(\mathbf{K})^\top\right)\mathbf{V} = \phi(\mathbf{Q})\left(\phi(\mathbf{K})^\top\mathbf{V}\right)$$

so we can reduce the amount of multiplications in calculating expression (3), by computing certain expression only once in in the beginning. How many multiplication are required for compute each row in a SA layer this method? You can neglect any multiplication in the denominator of Eq. (3) in this part and the next (why?).

7. Under which conditions the number of multiplications in part 6 is smaller than in part 4?

2. Optimization

We wish to optimize a loss function $\mathcal{L}(\mathbf{w})$, a twice differentiable function where $\mathbf{w}$ is a weight vector in $\mathbb{R}^d$ using Gradient Descent (GD) with some step size schedule $\eta_t > 0$

$$\forall t = 0, 1, .. : \mathbf{w}(t+1) = \mathbf{w}(t) - \eta_t \nabla \mathcal{L}(\mathbf{w}(t))$$

assume that $\mathcal{L}(\mathbf{w}) \geq 0$ and define

$$\beta(\mathbf{w}, \eta) \triangleq \max_{\alpha \in [0,1]} \lambda_{\max}\left(\nabla^2 \mathcal{L}(\mathbf{w} - \alpha\eta \nabla \mathcal{L}(\mathbf{w}))\right),$$

where $\lambda_{\max}$ denotes the maximal eigenvalue.

(1) Prove that if $\eta_t < \frac{2}{\beta(\mathbf{w}(t), \eta_t)}$, then the loss decreases monotonically at step t. Hint: From Taylor Expansion

$$\mathcal{L}(\mathbf{w}(t) - \eta_t \nabla \mathcal{L}(\mathbf{w}(t))) \leq \mathcal{L}(\mathbf{w}(t)) - \eta_t \|\nabla \mathcal{L}(\mathbf{w}(t))\|^2 + \frac{1}{2}\eta_t^2 \beta(\mathbf{w}(t), \eta_t) \|\nabla \mathcal{L}(\mathbf{w}(t))\|^2$$

From here forward, for simplicity we approximate $\beta(\mathbf{w}, \eta_t) \approx \beta(\mathbf{w}) = \lambda_{\max}(\nabla^2 \mathcal{L}(\mathbf{w}))$.

(2) Find a step size schedule η_t (as a function of $\beta(\mathbf{w}(t))$) for which we are guaranteed the fastest loss decrease at step t. Hint: Use the analysis from the previous part.

(3) Prove that if $\eta_t = \frac{1}{\beta(\mathbf{w}(t))}\left(1 - \sqrt{1 - \frac{1}{a_t}}\right)$ for some diverging series (for which $\sum_{t=0}^{\infty} a_t = \infty$) then we converge to a stationary point.

(4) Suppose $\beta = \max_{\mathbf{w} \in \mathbb{R}^d} \beta(\mathbf{w}) < \infty$. Do we have to decrease the step size for GD to converge to a stationary point? What about SGD? For what step schedule η_t are we guaranteed to converge for SGD (write an explicit mathematical condition)? No need to prove.

(5) Write two step-size schedules are typically used in practice when training using SGD.

3. "Classical" Generalization in Neural Networks

We examine binary classification problem on $\mathcal{S} = \left\{\mathbf{x}^{(n)}, y^{(n)}\right\}_{n=1}^{N}$, a dataset of samples, with each $\mathbf{x} \in \mathbb{R}^d$ and $y \in \{-1, 1\}$ sampled i.i.d. $(\mathbf{x}, y) \sim P_{X,Y}$, and a function $f \in \mathcal{F}$ that perfectly fits the data $\left(\forall n : f\left(\mathbf{x}^{(n)}\right) = y^{(n)}\right)$, where $\mathcal{F}$ is some pre-determined finite set functions (of size $|\mathcal{F}|$). In class we learned the following "classical" generalization bound:

Theorem. *For any $f \in \mathcal{F}$, with probability $1 - \delta$,*

$$\epsilon \triangleq \mathbb{P}_{(\mathbf{x},y)}\left(f\left(\mathbf{x}\right) \neq \boldsymbol{y}\right) < \frac{\log|\mathcal{F}| + \log 1/\delta}{N}$$

(1) Explain in words: what does ϵ mean?
(2) The Theorem says "with probability $1 - \delta$". What is the random variable (or process) over which we have this probability?
(3) Suppose $\mathcal{F}$ is the class of neural networks, with $k > N$ parameters, where each parameter is quantized to one of $q > 1$ values. Show this Theorem is not useful in this case.

Next, we aim prove the Theorem. We do this in three steps. Recall we defined

$$B_\epsilon\left(\mathcal{F}\right) = \left\{f \in \mathcal{F} : P_{X,Y}\left(f\left(\mathbf{x}\right) \neq y\right) > \epsilon\right\}$$

(4) Prove that $\forall f \in B_\epsilon\left(\mathcal{F}\right)$:

$$P_{\left\{\mathbf{x}^{(n)},y^{(n)}\right\}_{n=1}^{N} \overset{i.i.d.}{\sim} P_{X,Y}}\left(\forall n : f\left(\mathbf{x}^{(n)}\right) = y^{(n)}\right) < \left(1 - \epsilon\right)^N.$$

(5) Prove that (Hint: Union bound)

$$P_{\left\{\mathbf{x}^{(n)},y^{(n)}\right\}_{n=1}^{N} \overset{i.i.d.}{\sim} P_{X,Y}}\left(\exists f \in \mathcal{B}_\epsilon\left(\mathcal{F}\right) : \forall n : f\left(\mathbf{x}^{(n)}\right) = y^{(n)}\right) < e^{-\epsilon N}\left|\mathcal{F}\right|.$$

(6) Combine sections 4-5 together to obtain the Theorem.